

Desenvolvimento de um Compilador Didatico Baseado em Python com Interface Grafica

Arthur da Silva - 01622306

Gabriel Luan Soares de Oliveira - 01624195

Joao Victor Florencio - 01605737

Lucas Enthony Gomes Ferreira - 01576401

Miqueias Ferreira Barros - 01595460

Patrick Jose Viana Costa - 01594218

Junho de 2026

1 Introducao

Este trabalho apresenta o *hel.py*, um compilador didatico construido na linguagem C# com Windows Forms. O projeto foi desenvolvido com o objetivo de demonstrar, de forma pratica, conceitos de compiladores e interpretadores, como leitura de codigo fonte, reconhecimento de comandos, avaliacao de expressoes, controle de fluxo e tratamento de escopo.

A linguagem escolhida como referencia foi Python, por possuir uma sintaxe simples e bastante conhecida. Entretanto, o sistema nao busca implementar toda a linguagem Python original. O foco esta em um subconjunto didatico, suficiente para demonstrar comandos fundamentais como atribuicao de variaveis, saida de dados, expressoes aritmeticas, condicional `if else`, repeticao com `while` e blocos definidos por indentacao.

Como diferencial, o projeto possui uma interface grafica propria, permitindo que o usuario escreva o codigo, salve arquivos, abra exemplos, compile a estrutura do programa e execute o resultado dentro da propria aplicacao. Dessa forma, alem de cumprir o papel de compilador didatico, o *hel.py* tambem funciona como uma pequena IDE academica voltada para demonstracao em sala de aula.

2 Objetivos do Projeto

O objetivo principal do trabalho foi criar um compilador funcional, utilizando uma linguagem de programacao a escolha da equipe. A equipe optou por desenvolver o compilador didatico em C#, pois essa linguagem oferece boa integracao com Windows Forms e facilita a criacao de uma interface visual para entrada e saida de dados.

Entre os objetivos especificos, destacam-se:

- criar um compilador com editor, comandos de arquivo, console e area de compilacao;
- reconhecer comandos basicos inspirados em Python;

- implementar estrutura condicional com `if` e `else`;
- implementar estrutura de repeticao com `while`;
- validar blocos de codigo por meio de indentacao;
- executar expressoes aritmeticas com precedencia de operadores;
- exibir a saida do programa em um console interno;
- documentar o funcionamento do sistema em relatorio escrito em LaTeX;
- versionar o codigo e os arquivos do projeto em repositorio GitHub.

3 Tecnologias Utilizadas

O desenvolvimento foi realizado com a linguagem C# e a plataforma .NET. A interface grafica foi construida com Windows Forms, tecnologia adequada para aplicacoes desktop simples e para projetos academicos que precisam demonstrar eventos, formularios, botoes, caixas de texto e componentes visuais.

O relatorio foi produzido em LaTeX, utilizando uma estrutura tipografica simples com secoes, listas e trechos de codigo. Para versionamento, o projeto utiliza Git e foi preparado para entrega no GitHub, incluindo tanto o codigo-fonte do compilador quanto o arquivo do relatorio.

4 Arquitetura Geral do Sistema

O *hel.py* foi organizado em duas partes principais: a interface grafica e o motor de compilacao e execucao. A interface e responsavel pela interacao com o usuario, enquanto o motor faz a leitura, validacao, estruturacao e execucao das instrucoes escritas no editor.

Na interface, o usuario pode criar um novo arquivo, abrir arquivos com extensao `.py`, salvar o conteudo digitado e executar o codigo. O resultado da execucao e exibido em uma area de console, simulando a saida padrao de um programa.

O motor de compilacao recebe o texto do editor, divide o conteudo em linhas, remove comentarios, verifica a indentacao e identifica qual tipo de comando esta sendo processado. A partir disso, o sistema gera uma representacao logica do fluxo do programa e decide se deve realizar uma atribuicao, imprimir um valor, avaliar uma condicao, executar um bloco condicional ou repetir um bloco de codigo.

5 Subconjunto da Linguagem Compilada

A linguagem aceita pelo projeto e inspirada em Python, mas foi limitada aos comandos necessarios para cumprir o objetivo academico. A gramatica reconhecida pelo compilador inclui:

- atribuicoes numericas, como `x = 10`;
- atualizacao de variaveis, como `x = x + 1`;

- expressões aritméticas com `+`, `-`, `*` e `/`;
- uso de parênteses para alterar a precedência das operações;
- impressão de textos e valores com `print()`;
- comentários iniciados com `#`;
- estrutura condicional `if condicao::`;
- alternativa condicional `else::`;
- estrutura de repetição `while condicao::`;
- comparadores `>`, `<`, `>=`, `<=`, `==` e `!=`.

Esse conjunto permite construir algoritmos simples, mas completos o suficiente para demonstrar entrada lógica, tomada de decisão, repetição e manipulação de variáveis.

6 Processo de Compilação e Execução

O processo do *hel_py* combina uma etapa de compilação didática com uma etapa de execução. Na compilação, o código digitado pelo usuário é analisado, separado em linhas, validado e convertido em uma representação textual de comandos e desvios, semelhante a uma forma intermediária simplificada. Na execução, essa estrutura é percorrida para produzir o resultado no console da aplicação.

O código digitado pelo usuário não é transformado em um arquivo executável separado, como ocorre em compiladores profissionais completos. Mesmo assim, o projeto atende ao conceito acadêmico de compilador didático, pois analisa código-fonte, reconhece estruturas da linguagem, valida regras sintáticas, organiza o fluxo de execução e apresenta uma etapa de compilação antes da execução do programa.

O primeiro passo é separar o código em linhas. Em seguida, cada linha passa por uma limpeza básica, removendo comentários e ignorando espaços desnecessários. Depois disso, o compilador verifica a indentação da linha e identifica se ela pertence ao bloco atual.

Quando a linha representa uma atribuição, o sistema separa o nome da variável e a expressão à direita do sinal de igualdade. A expressão é calculada e o resultado é armazenado em uma tabela de símbolos. Essa tabela funciona como a memória do programa interpretado, guardando o valor atual de cada variável.

Quando a linha representa um `print()`, o sistema avalia o conteúdo informado entre parênteses. Caso seja um texto entre aspas, ele é exibido diretamente. Caso seja uma variável ou expressão, o valor é calculado antes de ser mostrado no console.

7 Avaliação de Expressões

Um dos pontos importantes do projeto foi a avaliação de expressões aritméticas. O compilador precisa respeitar a ordem correta das operações, evitando resultados incorretos. Por exemplo, a expressão `2 + 3 * 4` deve resultar em `14`, pois a multiplicação tem prioridade sobre a soma.

Para isso, a avaliação foi organizada em etapas, separando soma e subtração, multiplicação e divisão, valores entre parênteses, números e variáveis. Essa abordagem permite

que o sistema resolva expressões simples de maneira previsível e próxima do comportamento esperado em linguagens de programação reais.

O sistema também identifica erros comuns, como uso de variáveis não declaradas, divisão por zero ou expressões mal formadas. Quando ocorre um erro, a execução é interrompida e a mensagem é exibida na interface, ajudando o usuário a entender o problema.

8 Condicionais

A estrutura condicional foi implementada com suporte a `if` e `else`. O compilador avalia a condição informada após o `if`. Se o resultado for verdadeiro, o bloco indentado abaixo do `if` é executado. Caso contrário, se existir um bloco `else`, esse segundo bloco é executado.

Exemplo de código aceito pelo compilador:

```
vida = 0

if vida > 0:
    print("Personagem vivo")
else:
    print("Personagem derrotado")
```

Esse recurso atende ao requisito de tomada de decisão, permitindo que o programa execute caminhos diferentes de acordo com os valores das variáveis.

9 Estrutura de Repetição

Para o requisito de repetição, o projeto implementou o comando `while`. O compilador avalia a condição do `while` e executa o bloco indentado enquanto essa condição permanecer verdadeira.

Exemplo de código aceito:

```
x = 0
while x < 5:
    print(x)
    x = x + 1
```

Nesse exemplo, a variável `x` é incrementada a cada repetição. O laço termina quando `x` deixa de ser menor que 5. Esse comportamento demonstra o uso de variáveis, comparadores, expressões e repetição em conjunto.

Também foi implementado um limite de segurança para evitar repetições infinitas. Caso um programa possua uma condição que nunca se torna falsa, a execução é interrompida automaticamente após determinado número de iterações.

10 Validação de Escopo por Indentação

Ao contrário de linguagens que delimitam blocos com chaves, Python utiliza recuo textual. O motor desenvolvido calcula a quantidade de espaços antes do primeiro caractere válido de cada linha. Quando encontra comandos que iniciam blocos, como `if` e `while`, o

compilador chama recursivamente a rotina de processamento para o nível seguinte de indentacao.

Essa estrategia permite isolar blocos internos e encerrar a execucao do escopo atual quando a indentacao retorna ao nível anterior. O projeto adotou quatro espacos como padrao de indentacao para facilitar a validacao e a demonstracao.

11 Interface Grafica

A interface foi desenvolvida com Windows Forms. A janela principal possui um editor de codigo a esquerda, um console de saida a direita, uma area de compilacao e um painel superior com comandos para criar, abrir, salvar e executar arquivos Python. Esse conjunto de recursos faz com que o compilador tenha uma experiencia parecida com uma mini IDE, facilitando a escrita e os testes dos algoritmos.

A escolha por uma interface visual teve como finalidade tornar a demonstracao mais clara. Em vez de executar o compilador apenas pelo terminal, o usuario consegue visualizar o codigo, a saida e as mensagens de compilacao em uma mesma janela. Isso facilita a apresentacao para a turma e deixa evidente o funcionamento de cada etapa.

12 Tratamento de Erros

Durante o desenvolvimento, foram considerados erros comuns que poderiam ocorrer ao escrever codigo no editor. O sistema foi preparado para emitir mensagens quando encontra problemas como:

- comando nao reconhecido;
- uso incorreto de indentacao;
- `else` sem `if` correspondente;
- variavel utilizada antes de receber valor;
- expressao aritmetica invalida;
- divisao por zero;
- repeticao interrompida por limite de seguranca.

Essas mensagens sao importantes porque aproximam o projeto do comportamento de ferramentas reais de desenvolvimento, nas quais o programador precisa receber retorno claro sobre os erros do codigo.

13 Testes e Exemplos

Foram criados arquivos de exemplo para validar o funcionamento do compilador. O arquivo `demo.py` demonstra o uso de `while`, `if`, `else`, impressao e incremento de variavel. O arquivo `if_else.py` apresenta um exemplo menor, focado apenas na estrutura condicional. Ja o arquivo `combate_chefe.py` utiliza uma situacao mais elaborada, combinando laço, condicoes e calculos.

Os testes foram feitos executando os exemplos na propria interface do sistema e verificando se a saida exibida no console correspondia ao comportamento esperado. Tambem foi realizada compilacao do projeto pelo .NET para confirmar que o codigo C# nao possuia erros de construcao.

14 Versionamento e Organizacao da Entrega

O projeto foi versionado com Git e preparado para entrega em repositorio GitHub. O versionamento inclui o codigo-fonte do compilador, os exemplos de teste, os recursos visuais da interface e o relatorio em LaTeX.

A organizacao em repositorio permite acompanhar a evolucao do trabalho, registrar alteracoes realizadas e facilitar a colaboracao entre os integrantes. Para a entrega final, o repositorio deve conter tanto o compilador quanto o relatorio, conforme solicitado nas regras da apresentacao.

15 Participacao da Equipe

O desenvolvimento foi conduzido de forma colaborativa, envolvendo pesquisa, implementacao, testes e ajustes de interface. A equipe trabalhou na definicao do escopo da linguagem, na escolha das funcionalidades essenciais, na construcao dos exemplos e na revisao do comportamento do compilador.

As principais decisoes tecnicas foram tomadas manualmente pela equipe, como a escolha de C# com Windows Forms, a adocao de uma sintaxe inspirada em Python, o uso de quatro espacos para indentacao, a separacao entre interface e motor de interpretacao e a limitacao proposital da linguagem para fins academicos.

16 Apoio de Inteligencia Artificial

A inteligencia artificial foi utilizada apenas como ferramenta de apoio em momentos especificos do desenvolvimento, principalmente para revisar ideias, sugerir organizacao de texto, auxiliar na identificacao de possiveis erros e melhorar a clareza de algumas mensagens. A implementacao, os testes, as escolhas de tecnologia e a definicao do funcionamento principal foram conduzidos pela equipe.

Um exemplo de ajuste realizado durante o desenvolvimento foi a correcao do comportamento de atribuicoes como `x = x + 1`, necessarias para que lacos `while` funcionassem corretamente. Outro ponto revisado foi a inclusao de um limite de seguranca para repeticoes, evitando que programas com condicoes sempre verdadeiras deixassem a aplicacao presa.

Assim, o uso de IA nao substituiu o trabalho da equipe. Ele serviu como apoio complementar, semelhante a uma consulta tecnica, enquanto a construcao, validacao e apresentacao do projeto permaneceram sob responsabilidade dos integrantes.

17 Linguagem Tecnica e Explicacao Simplificada

Em termos tecnicos, o *hel.py* pode ser descrito como um compilador didatico para uma linguagem simplificada baseada em Python, apresentado dentro de uma interface grafica

com características de mini IDE. Ele possui uma tabela de símbolos para armazenar variáveis, um avaliador de expressões aritméticas, um avaliador de condições booleanas e um mecanismo de controle de escopo por indentação.

De forma mais simples, o programa funciona como um compilador visual: o usuário escreve um código, manda compilar e executa o resultado na própria tela. O sistema analisa cada linha, entende se ela representa uma variável, uma impressão, uma condição ou uma repetição, e executa a ação correspondente. Quando encontra um bloco indentado, ele entende que aquele trecho pertence ao comando anterior, como acontece em Python.

18 Conclusão

A construção do *hel.py* consolidou conceitos importantes de compiladores e interpretadores, como análise de linhas, reconhecimento de comandos, avaliação de expressões, manipulação de variáveis, controle de fluxo e validação de escopo. O projeto também permitiu aplicar conhecimentos de interface gráfica, organização de código, tratamento de erros e versionamento, resultando em um compilador didático mais completo por estar integrado a uma interface de mini IDE.

Embora a linguagem interpretada seja limitada, ela atende aos requisitos principais da atividade ao oferecer suporte a condicional `if else`, repetição com `while`, expressões aritméticas e execução de comandos em uma interface própria. Dessa forma, o trabalho demonstra de maneira prática como uma aplicação em C# pode interpretar instruções inspiradas em Python e transformar texto escrito pelo usuário em comportamento executável.